Panorama da Banda Larga em Alagoas

Fase 3 · Dashboard Streamlit · Planejamento e guia de implementação · 04/11/2026 · Gabriel

Objetivo da fase

Transformar as 6 views analíticas do MySQL num dashboard interativo, com filtros de período e de operadora em destaque, que conte a história do mercado de banda larga fixa de Alagoas. Duração prevista: **2h30**, em 7 etapas com 2 paradas.

Onde o projeto está

Etapa	Situação
Exploração, dicionário de dados e recorte 2021+	Concluída
Esquema estrela no MySQL	Concluída
Pipeline ETL (extract, transform, load, checkpoint)	Concluída
6 views analíticas (02_views.sql), criadas pelo pipeline	Concluída
Validações (03_validacoes.sql)	Concluída
Dashboard Streamlit	**Em andamento (esta fase)**
Power BI (versão para o Whebert)	Pendente
README, teste do zero e envio	Pendente

Decisões de arquitetura desta fase

- **Carregar cada view inteira uma vez e filtrar no pandas.** São poucos milhares de linhas; com o cache, cada clique num filtro responde na hora, sem voltar ao banco.
- **As views são o contrato.** O dashboard só lê das views; toda regra de negócio continua no SQL.
- **Filtro por CNPJ, nunca por nome.** O seletor mostra o nome, mas trabalha com o CNPJ (a Claro tem 2).
- **Arredondar uma vez só.** As views já entregam 2 casas; o dashboard não arredonda de novo.
- **Transparência.** Seções que não acompanham o filtro de período dizem isso na tela; o rodapé lista as limitações da base.

Cronograma

Etapa	Conteúdo	Tempo	Acumulado
1	Reorganizar o código, carregar as views, filtros e KPIs	30 min	0h30
2	Gráfico de evolução do mercado	15 min	0h45
3	Gráfico da virada da fibra	15 min	1h00
Parada 1	Commit e 5 min de pausa		
4	Market share com a operadora em destaque	25 min	1h25
5	Painel da operadora em destaque	25 min	1h50
Parada 2	Commit e 5 min de pausa		

Etapa	Conteúdo	Tempo	Acumulado
6	Tabela dos municípios	20 min	2h10
7	Lacunas de fibra (Starlink) e rodapé	15 min	2h25
Final	Commit final e prints para o README	5 min	2h30

Se o tempo apertar, a Etapa 7 é a primeira a ir para o dia seguinte.

Conceitos-chave do Streamlit

- **O script roda inteiro a cada interação.** Cada clique num filtro reexecuta o arquivo de cima para baixo; os widgets devolvem o valor atual.
- **@st.cache_resource** guarda o que deve existir uma vez só (a conexão com o banco).
- **@st.cache_data** guarda resultados (as tabelas das consultas).
- **Rodar:** `streamlit run app/app.py` na raiz, com o (.venv) ativo. Ative “Always rerun” para atualizar a cada Ctrl + S. Encerrar: Ctrl + C no terminal.
- **Compatibilidade:** se `width="stretch"` der erro, a versão do Streamlit é antiga: use `use_container_width=True`.

Etapa 1 · Reorganizar, carregar os dados e criar os filtros (30 min)

Substitua o `app/app.py` inteiro por este código. Ele já inclui a correção da conversão de tipos (ver "Erros já resolvidos" no final).

1.1 Imports e funções auxiliares

```
"""Dashboard: Panorama da Banda Larga Fixa em Alagoas."""
import math
import sys
from decimal import Decimal
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

# O Streamlit roda a partir de app/: avisamos onde fica a raiz do projeto.
# Esta linha PRECISA vir antes do "from src.db import ..."
sys.path.append(str(Path(__file__).resolve().parent.parent))
from src.db import get_engine  # noqa: E402

st.set_page_config(page_title="Banda Larga em Alagoas", page_icon=":signal_strength:", layout="wide")

COR_DESTAQUE = "#E4572E"
COR_NEUTRA = "#B0B7C3"


@st.cache_resource
def conexao():
    """Cria a conexão com o banco uma única vez."""
    return get_engine()


@st.cache_data(ttl=3600)
def consultar(sql: str) -> pd.DataFrame:
    """Executa uma consulta, ajusta os tipos e guarda o resultado por 1 hora."""
    df = pd.read_sql(sql, conexao())
    # Só as colunas DECIMAL viram float. CNPJ e código IBGE continuam texto.
    for coluna in df.columns:
        valores = df[coluna].dropna()
        if not valores.empty and isinstance(valores.iloc[0], Decimal):
            df[coluna] = df[coluna].astype(float)
    if "ano_mes" in df.columns:
        df["ano_mes"] = pd.to_datetime(df["ano_mes"])
    return df


def br(numero, casas: int = 0) -> str:
    """Formata um número no padrão brasileiro: 1.234,5"""
    if pd.isna(numero):
        return "–"
    texto = f"{numero:,.{casas}f}"
    return texto.replace(",", "X").replace(".", ",").replace("X", ".")


def fmt_cnpj(cnpj: str) -> str:
    """02558157000162 -> 02.558.157/0001-62"""
    return f"{cnpj[:2]}.{cnpj[2:5]}.{cnpj[5:8]}/{cnpj[8:12]}-{cnpj[12:]}"


def mes(data) -> str:
    """Data -> 'mm/aaaa'"""
    return pd.Timestamp(data).strftime("%m/%Y")


def estilo(fig, altura: int = 350):
    """Aplica o mesmo visual a todos os gráficos."""
    fig.update_layout(
        height=altura,
        margin=dict(l=0, r=0, t=10, b=0),
        separators=",.",              # decimal com vírgula, milhar com ponto
        hovermode="x unified",
```

```
        legend=dict(orientation="h", y=-0.15, title=None),
    )
    return fig
```

Como funciona: o laço do `consultar` só converte as colunas cujo valor é um `Decimal` (as que passaram por ROUND no SQL). A função `estilo` aplica o princípio DRY aos gráficos; o `separators=",."` faz o Plotly usar o padrão brasileiro.

1.2 Dados e filtros

```
# Dados: cada view é lida uma vez e fica em cache
evolucao   = consultar("SELECT * FROM vw_evolucao_mensal")
mercado    = consultar("SELECT * FROM vw_market_share")
tecnologia = consultar("SELECT * FROM vw_tecnologia")
destaque   = consultar("SELECT * FROM vw_operadora_destaque")
municipios = consultar("SELECT * FROM vw_municipio_resumo")
starlink   = consultar("SELECT * FROM vw_starlink_municipio")

meses = list(evolucao["ano_mes"].sort_values())

# Barra lateral
st.sidebar.header("Filtros")
inicio, fim = st.sidebar.select_slider(
    "Período", options=meses, value=(meses[0], meses[-1]), format_func=mes)

# Operadoras do último mês do período, da maior para a menor
no_fim = mercado[mercado["ano_mes"] == fim].sort_values("ranking").copy()

# Nomes repetidos (ex.: os dois CNPJs da Claro) ganham o CNPJ no rótulo
repetidos = no_fim["empresa"].duplicated(keep=False)
no_fim["rotulo"] = no_fim["empresa"].where(
    ~repetidos, no_fim["empresa"] + " (" + no_fim["cnpj"].map(fmt_cnpj) + ")")
rotulos = dict(zip(no_fim["cnpj"], no_fim["rotulo"]))

cnpj_destaque = st.sidebar.selectbox(
    "Operadora em destaque", options=list(rotulos), format_func=rotulos.get,
    help="Ordenadas por número de acessos no último mês do período")
nome_destaque = rotulos[cnpj_destaque]

st.sidebar.caption("Fonte: dados abertos da Anatel · banda larga fixa (SCM), produto INTERNET")


def no_periodo(df: pd.DataFrame) -> pd.DataFrame:
    """Mantém só as linhas dentro do período escolhido."""
    return df[df["ano_mes"].between(inicio, fim)]
```

- **select_slider com dois valores** cria uma barra com duas alças e devolve (início, fim). O `format_func` muda só o que aparece na tela.
- **duplicated(keep=False) + where**: onde o nome não se repete, mantém o nome; onde se repete, acrescenta o CNPJ formatado.
- **selectbox** trabalha com o CNPJ como valor e mostra o rótulo na tela.
- **no_periodo** é definida depois dos filtros porque usa `inicio` e `fim`.

1.3 Cabeçalho e KPIs

```
st.title(":signal_strength: Panorama da Banda Larga Fixa em Alagoas")
st.caption(f"Período analisado: {mes(inicio)} a {mes(fim)}")

linha_fim = evolucao[evolucao["ano_mes"] == fim].iloc[0]
fibra = (tecnologia[tecnologia["meio_acesso"] == "Fibra"]
         .set_index("ano_mes")["participacao_pct"])
lider = no_fim.iloc[0]

col1, col2, col3, col4 = st.columns(4)
col1.metric(
    f"Acessos em {mes(fim)}",
    br(linha_fim["total_acessos"]),
    None if pd.isna(linha_fim["var_pct_12m"])
    else f"{br(linha_fim['var_pct_12m'], 1)}% em 12 meses")
col2.metric("Operadoras ativas", br(len(no_fim)))
col3.metric(
    "Participação da fibra",
```

```
    f"{br(fibra[fim], 2)}%",
    f"{br(fibra[fim] - fibra[inicio], 2)} p.p. desde {mes(inicio)}")
col4.metric(
    "Líder do mercado", lider["rotulo"],
    f"{br(lider['participacao_pct'], 1)}% do mercado", delta_color="off")

st.divider()
```

Checkpoint 1: a barra lateral tem os dois filtros e os 4 KPIs mudam quando o período muda. Com o período completo: 382.217 acessos, 295 operadoras, 82,35% de fibra, Brisanet líder.

Etapa 2 · Evolução do mercado (15 min)

Adicione no final do arquivo:

```
st.subheader("1. Evolução do mercado")

fig = px.line(no_periodo(evolucao), x="ano_mes", y="total_acessos", markers=True,
              labels={"ano_mes": "", "total_acessos": "Acessos"})
fig.update_traces(line_color=COR_DESTAQUE, hovertemplate="%{y:,.0f} acessos")
st.plotly_chart(estilo(fig), width="stretch")
```

- **px.line** recebe a tabela e o nome das colunas dos eixos; o Plotly Express cuida do resto.
- **labels** troca nomes técnicos por nomes legíveis. **hovertemplate** controla o texto do mouse: `%{y:,.0f}` = valor de Y com milhar e sem decimais.

Checkpoint 2: a linha mostra o mercado mês a mês e encolhe quando o período é reduzido.

Etapa 3 · A virada da fibra (15 min)

```
st.subheader("2. A virada da fibra")

fig = px.area(
    no_periodo(tecnologia), x="ano_mes", y="participacao_pct", color="meio_acesso",
    category_orders={"meio_acesso": ["Fibra"]},   # fibra sempre primeiro
    labels={"ano_mes": "", "participacao_pct": "% dos acessos", "meio_acesso": ""})
fig.update_traces(hovertemplate="%{y:.1f}%")
fig.update_yaxes(range=[0, 100], ticksuffix="%")
st.plotly_chart(estilo(fig), width="stretch")

st.caption(
    f"Em {mes(inicio)}, a fibra respondia por {br(fibra[inicio], 2)}% dos acessos; "
    f"em {mes(fim)}, por {br(fibra[fim], 2)}%.")
```

- **px.area com color=** empilha uma área por meio de acesso; como as participações somam 100%, dá para ver a fibra ocupando o espaço das outras tecnologias.
- **A legenda é gerada pelos dados**: nada de número fixo no texto (regra R3 no dashboard).

Checkpoint 3: a área da fibra cresce ao longo do tempo e a frase acompanha o período.

PARADA 1 (1h00). `git add . && git commit -m "feat: dashboard com filtros, evolução e tecnologia"` · 5 min de pausa.

Etapa 4 · Market share com destaque (25 min)

```
st.subheader(f"3. Quem lidera o mercado ({mes(fim)})")

top = no_fim.head(10).copy()
top["grupo"] = top["cnpj"].eq(cnpj_destaque).map({True: "Em destaque", False: "Demais"})

fig = px.bar(
    top, x="participacao_pct", y="rotulo", orientation="h", color="grupo",
    color_discrete_map={"Em destaque": COR_DESTAQUE, "Demais": COR_NEUTRA},
    text="participacao_pct", labels={"participacao_pct": "% do mercado", "rotulo": ""})
fig.update_traces(texttemplate="%{text:.1f}%", textposition="outside",
                  hovertemplate="%{x:.2f}%")
fig = estilo(fig, altura=420)
fig.update_layout(hovermode="closest", showlegend=False,
                  yaxis={"categoryorder": "total ascending"})
```

```
st.plotly_chart(fig, width="stretch")

# Se a operadora escolhida não está no top 10, avisamos onde ela está
if cnpj_destaque not in set(top["cnpj"]):
    posicao = no_fim[no_fim["cnpj"] == cnpj_destaque].iloc[0]
    st.info(
        f"**{nome_destaque}** está fora do top 10: {int(posicao['ranking'])}º lugar, "
        f"com {br(posicao['participacao_pct'], 2)}% do mercado.")
```

- **Cinza para o contexto, cor para o que importa:** a coluna `grupo` tem só dois valores, e o `color_discrete_map` pinta a escolhida de laranja.
- **categoryorder: total ascending** mantém as barras ordenadas por tamanho mesmo com duas cores.
- **Todo filtro precisa dar uma resposta visível:** o aviso azul cobre operadoras fora do top 10.

Checkpoint 4: top 10 com a escolhida em laranja; uma operadora pequena gera o aviso azul.

Etapa 5 · Painel da operadora em destaque (25 min)

```
st.subheader(f"4. Em destaque: {nome_destaque}")

op = no_periodo(destaque[destaque["cnpj"] == cnpj_destaque]).sort_values("ano_mes")

# Grade com TODOS os meses do período: meses sem reporte viram buracos no gráfico
meses_periodo = pd.DataFrame({"ano_mes": [m for m in meses if inicio <= m <= fim]})
serie = meses_periodo.merge(op, on="ano_mes", how="left")

atual = op[op["ano_mes"] == fim].iloc[0]
primeiro = op.iloc[0]

c1, c2, c3 = st.columns(3)
c1.metric("Acessos", br(atual["acessos_totais"]),
          f"{br(atual['acessos_totais'] - primeiro['acessos_totais'])} desde {mes(primeiro['ano_mes'])}")
c2.metric("Participação", f"{br(atual['participacao_pct'], 2)}%",
          f"{br(atual['participacao_pct'] - primeiro['participacao_pct'], 2)} p.p.")
c3.metric("Ranking", f"{int(atual['ranking'])}º",
          f"{int(primeiro['ranking'] - atual['ranking'])} posições desde {mes(primeiro['ano_mes'])}")

g1, g2 = st.columns(2)

fig = px.line(serie, x="ano_mes", y="acessos_totais", markers=True,
              labels={"ano_mes": "", "acessos_totais": "Acessos"})
fig.update_traces(line_color=COR_DESTAQUE, hovertemplate="%{y:,.0f} acessos")
g1.plotly_chart(estilo(fig, altura=300), width="stretch")

fig = px.line(serie, x="ano_mes", y="participacao_pct", markers=True,
              labels={"ano_mes": "", "participacao_pct": "% do mercado"})
fig.update_traces(line_color=COR_DESTAQUE, hovertemplate="%{y:.2f}%")
g2.plotly_chart(estilo(fig, altura=300), width="stretch")

# A pegadinha do Card 4, agora visível para quem usa o dashboard
meses_sem_reporte = int((op["meses_ultima_aparicao"] - 1).clip(lower=0).sum())
if meses_sem_reporte:
    st.warning(
        f"{nome_destaque} ficou **{meses_sem_reporte} mês(es) sem reportar** acessos no período. "
        "Esses intervalos aparecem como falhas na linha.")
```

- **merge(..., how="left") com a grade de meses:** os meses sem dado viram NaN e o Plotly interrompe a linha. O buraco do reporte vira um buraco visível.
- **primeiro = op.iloc[0]** é o primeiro mês com dados da operadora dentro do período.
- **Ranking invertido (primeiro − atual):** número menor é melhor, então subir de posição fica verde.
- **Meses sem reporte:** distância 4 = 3 meses faltando, daí o − 1; o clip zera as distâncias normais.

Checkpoint 5: a Claro mostra linhas contínuas; a operadora com o buraco de 40 e poucos meses mostra a falha na linha e o aviso amarelo.

PARADA 2 (1h50). `git add . && git commit -m "feat: market share e painel da operadora em destaque"` · 5 min de pausa.

Etapa 6 · Tabela dos municípios (20 min)

```
st.subheader(f"5. Os municípios ({mes(meses[-1])})")
st.caption(
    "Retrato do último mês disponível: esta seção não acompanha o filtro de período. "
    "HHI vai de 0 a 10.000; acima de 2.500, o mercado é considerado altamente concentrado. "
    "Concorrentes equivalentes = 10.000 ÷ HHI.")

tabela = (municipios
          .assign(concorrentes_equivalentes=10_000 / municipios["hhi"])
          .sort_values("acessos_municipio", ascending=False))

busca = st.text_input("Buscar município", placeholder="Ex.: Arapiraca")
if busca:
    tabela = tabela[tabela["municipio"].str.contains(busca, case=False, na=False)]

st.dataframe(
    tabela[["municipio", "acessos_municipio", "operadoras", "empresa_lider",
            "participacao_lider_pct", "fibra_pct", "hhi", "concorrentes_equivalentes"]],
    hide_index=True, width="stretch", height=400,
    column_config={
        "municipio": "Município",
        "acessos_municipio": st.column_config.NumberColumn("Acessos", format="localized"),
        "operadoras": "Operadoras",
        "empresa_lider": "Líder",
        "participacao_lider_pct": st.column_config.NumberColumn("Part. do líder", format="%.1f%%"),
        "fibra_pct": st.column_config.ProgressColumn(
            "Fibra", format="%.1f%%", min_value=0, max_value=100),
        "hhi": st.column_config.NumberColumn("HHI", format="localized"),
        "concorrentes_equivalentes": st.column_config.NumberColumn(
            "Concorrentes equiv.", format="%.1f"),
    })
```

- **Transparência:** a legenda avisa que a seção não acompanha o filtro de período.
- **assign** cria a coluna de concorrentes equivalentes (10.000 ÷ HHI) no pandas.
- **str.contains(case=False)** filtra pelo texto digitado (limitação: “maceio” não acha “Maceió”).
- **column_config** dá nomes legíveis, formatos e uma barra de progresso na coluna de fibra.

Checkpoint 6: a busca por “Maceió” mostra 128 operadoras e cerca de 3,7 concorrentes equivalentes.

Etapa 7 · Lacunas de fibra e rodapé (15 min)

```
st.subheader(f"6. Lacunas de fibra: onde a Starlink é mais forte ({mes(meses[-1])})")

ordenado = starlink.sort_values("participacao_starlink_pct", ascending=False)
n_quartil = math.ceil(len(ordenado) / 4)      # 98 municípios -> 25
topo, resto = ordenado.head(n_quartil), ordenado.iloc[n_quartil:]

c1, c2, c3 = st.columns(3)
c1.metric("Municípios com Starlink", f"{len(starlink)} de {len(municipios)}")
c2.metric(f"Fibra nos {n_quartil} onde a Starlink é mais forte",
          f"{br(topo['fibra_pct'].mean(), 1)}%")
c3.metric("Fibra nos demais municípios", f"{br(resto['fibra_pct'].mean(), 1)}%")

st.dataframe(
    topo[["municipio", "acessos_starlink", "participacao_starlink_pct", "fibra_pct"]],
    hide_index=True, width="stretch",
    column_config={
        "municipio": "Município",
        "acessos_starlink": st.column_config.NumberColumn("Acessos Starlink", format="localized"),
        "participacao_starlink_pct": st.column_config.NumberColumn(
            "Part. da Starlink", format="%.1f%%"),
        "fibra_pct": st.column_config.ProgressColumn(
            "Fibra", format="%.1f%%", min_value=0, max_value=100),
    })

# Rodapé: limitações
```

```
st.divider()
st.caption(
    "**Limitações.** Os dados são os acessos reportados pelas operadoras à Anatel: "
    "algumas reportam de forma intermitente, e há subnotificação, principalmente entre "
    "pequenos provedores. As análises consideram apenas o produto INTERNET e o período "
    "de jan/2021 em diante, após a mudança de metodologia da Anatel.")
```

- **math.ceil(len / 4)** arredonda para cima (98 ÷ 4 = 24,5 → 25), o mesmo tamanho do primeiro grupo do NTILE(4) no SQL. Dashboard e teste da hipótese mostram os mesmos números.
- **Os três KPIs contam a história**; a tabela é o mapa de lacunas de fibra.
- **O rodapé de limitações** diz o que os dados não mostram, antes que alguém conclua errado.

Checkpoint 7: cerca de 55% de fibra nos 25 municípios e cerca de 90% nos demais.

FINAL (2h30). `git add . && git commit -m "feat: dashboard completo" && git push` · tire prints de cada seção para o README.

Erros já resolvidos nesta fase

Erro	Causa	Correção
ModuleNotFoundError: No module named 'src'	O import de src vinha antes do sys.path.append (ou a linha sumiu ao colar).	sys.path.append(...) imediatamente antes de from src.db import get_engine.
TypeError: 'int' object is not subscriptable (em fmt_cnpj)	A primeira versão do consultar convertia para número toda coluna que conseguisse, inclusive CNPJ e código IBGE, apagando os zeros à esquerda.	Converter só as colunas cujo valor é Decimal (versão deste documento).

Lição: a mesma armadilha dos zeros à esquerda do CNPJ aparece em qualquer camada (CSV, banco ou pandas). Converter tipos “no atacado” é perigoso; converta só o que você sabe que é número.

Próximos passos

- Power BI: versão do painel para o Whebert, conectada às mesmas views (~2h).
- README com prints, os 5 insights e a seção de limitações (~1h).
- Teste do zero: clonar o repositório, rodar o pipeline e o dashboard seguindo só o README (~30 min).
- Opcionais: Cards 7, 8 e 9; view base para os Cards 5 e 6; coluna de concorrentes equivalentes na view.